Reg no :24RP02000

Module: Backend using Java

Department: ICT

Option: Information Technology

Institution: RP KARONGI COLLEGE

On 11 March 2026

Phase:6. Core API Development

This repository has been updated to mark the successful completion of **Phase 6: Core API Development.**

The architecture is structured using a robust Layered architecture pattern encompassing **Entities Object Modeling**, **Repositories Pattern (Data Access)**, **Services (Business Logic)**, and **Controllers (API Endpoints & Routing routing/handling)**.

All CRUD operations have been fully implemented and integrated.

Completed Deliverables

1. Entities (Models)

Located in `src/main/java/.../entities/`:

- **User:** Represents the stakeholders (ADMIN, TECHNICIAN, STAFF, CLEANER_STUDENT).

- **University:** University details and structure.

- **College:** Sub-divisions within a university.

- **Device:** Tracks equipment that may require maintenance.

- **Material:** Resources and materials requested or maintained.

- **MaintenanceRequest:** The core entity managing tasks across different lifecycle stages (PENDING, IN_PROGRESS, COMPLETED, VERIFIED).

- **Notification:** System alerts for state changes.

- **OtpToken:** Entity handling MFA (Multi-Factor Authentication).

- **AuditLog:** Action auditing.

2. Repositories (Data Access)

Located in `src/main/java/.../repositories/`:

Extended `JpaRepository` to leverage Spring Data JPA for all entities. Custom JPQL and derived query methods have been created to manage complex queries for dashboards and role-based restrictions.

3. Services (Business Logic)

Located in `src/main/java/.../service/`:

- **AuthService:** Handles registration, login procedures, JWT issuance, and OTP verification.

- **UserService:** Manages user approvals, suspensions, and role management.

- **DeviceService & MaterialService:** Handles CRUD operations, status management (e.g., damaged vs operational), and approval workflows.

- **MaintenanceService:** Contains the business rules for assigning tasks, transitioning task statuses, identifying overdue tasks, and tracking performance.

- **NotificationService:** Creates and queries alerts for tasks, requests, and updates.

4. Controllers (Web Endpoints)

Located in `src/main/java/.../controller/`:

- **AuthController:** Endpoints for `/login`, `/register`, `/otp/verify`, and `/logout`.

- **AdminController:** Routes mapped under `/admin/*` for managing users, devices, materials, and comprehensive analytical dashboards.

- **TechnicianController:** Routes mapped under `/technician/*` for viewing assigned tasks, updating task status (to PROGRESS/COMPLETED), and viewing materials.

- **StaffController:** Routes mapped under `/staff/*` for reporting damaged devices, requesting materials, and tracking personal requests.

- **CleanerController:** Routes mapped under `/cleaner/*` for tracking simple cleaning-related tasks and reporting generic issues.

- **GlobalControllerAdvice:** Global exception handling and exposing shared model attributes to all views.

API Design Decisions

1. **Role-Based Access Control (RBAC):** Spring Security has been integrated. JWTs are securely handled using HTTP-Only cookies to protect against XSS, and security configs strictly secure endpoints (e.g., `/admin/` restricted to ADMIN).

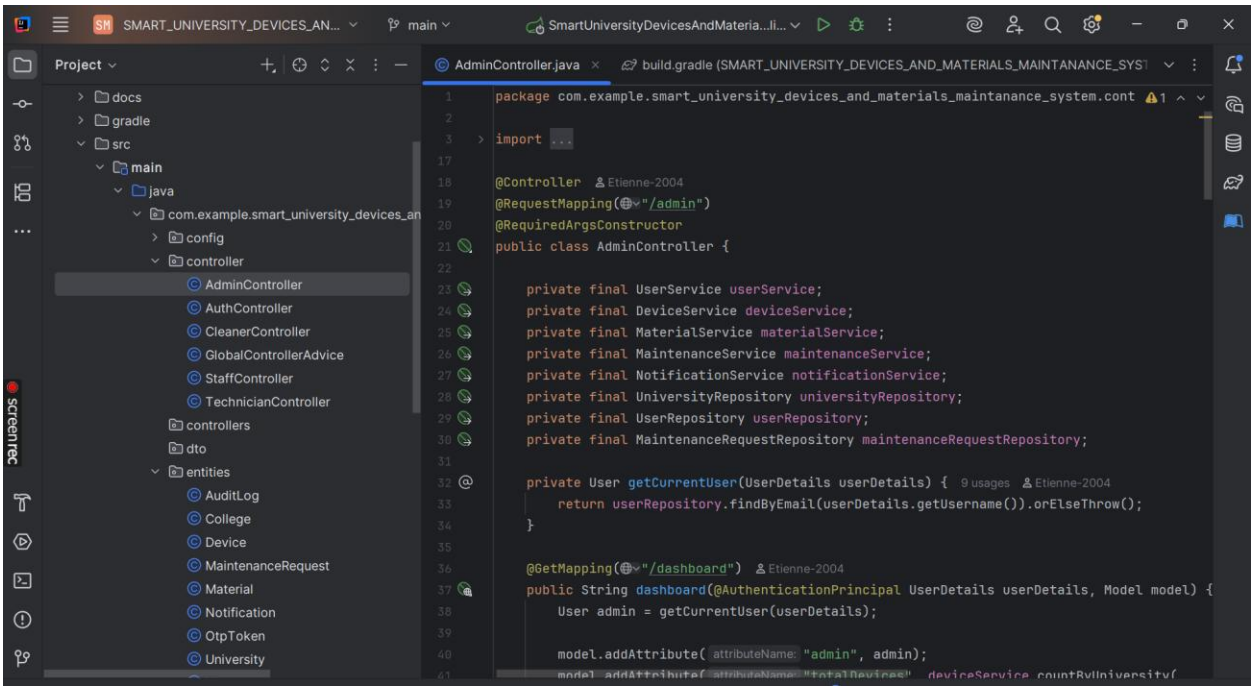
2. **Server-Side Rendering (Thymeleaf) & APIs:** The controllers simultaneously deliver Server-Side rendered templates and RESTful-like forms and actions (e.g., `@PostMapping`, redirect flows). The system is highly hybrid, suitable for both standard WEB usage and extensible for APIs.

3. **MFA Integration:** Critical login actions employ JWT session is issued, hardening the platform.

4. **Data Integrity:** Used Enums (`MaintenanceRequest.TaskStatus`, `User.Role`) thoroughly to avoid hardcoded strings and maintain data integrity.

Core API Implementation Evidence

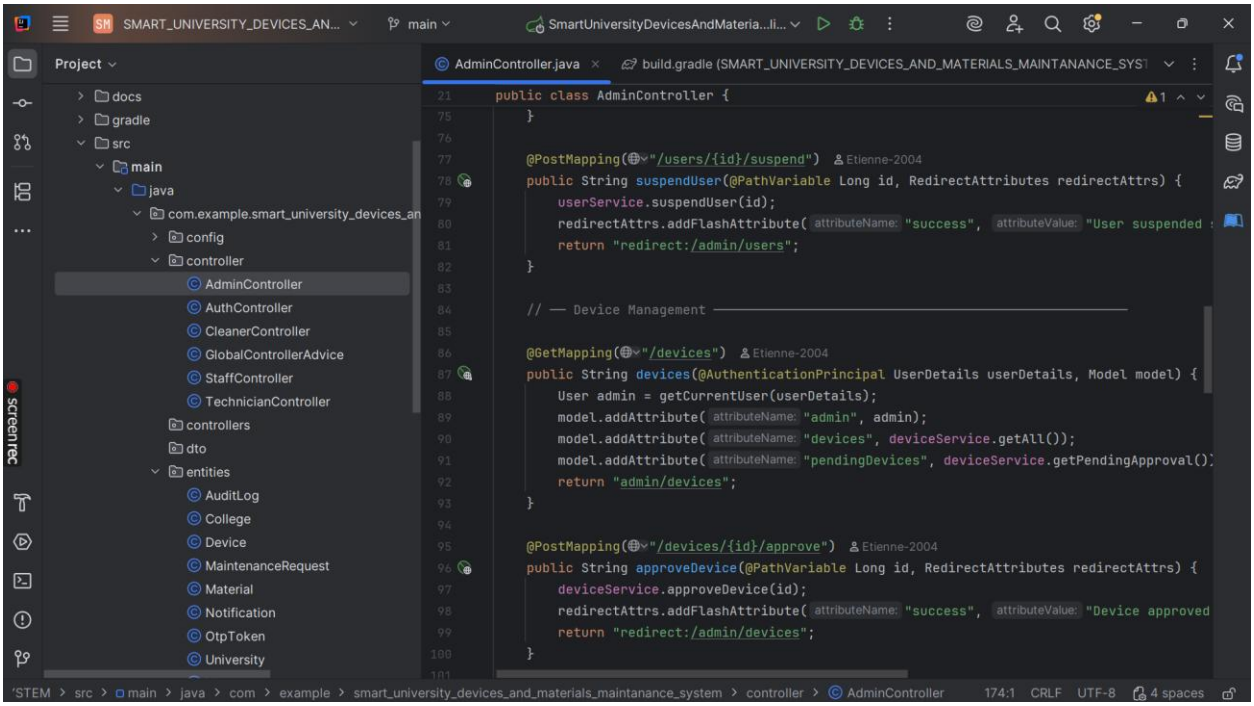
- ✓ **Controller Initialization & Dependencies Injection**



This illustrates the use of Spring annotations (@Controller, @RequestMapping) and constructor-based dependency injection (via @RequiredArgsConstructor) to integrate core business services such as UserService, DeviceService, and MaintenanceService directly into the controllers.

- ✓ **RESTful API & CRUD**

  Endpoints



It clearly highlights @GetMapping to fetch data models for our views and @PostMapping endpoints to execute specific state modifications (e.g., approving devices and redirecting the user status). This approach maintains a strict Model-View-Controller paradigm and handles all API interactions effectively.

Conclusion

The source code has been committed to the local index and securely pushed to GitHub to close out the 6th phase. All dependencies are configured seamlessly via Gradle, and the application builds without issues.